

FORMACIÓN DE GIT

Proyecto: CTTI

Nogueras Pardo, Francisco Javier <fjnogueras@indra.es> - 14/12/2017

Índice del contenido

¿Qué vamos a ver hoy?.....	3
De dónde viene GIT.....	3
OBJETIVOS:.....	3
GIT vs SVN.....	4
¿Cambio de mentalidad?.....	5
Distribuido.....	5
Tenemos que trabajar con control de versiones.....	5
Git no es SVN.....	6
¿Tan difícil es cambiar de subversion a git?.....	6
Estados de git.....	7
¿Qué clientes existen?.....	8
Principales comandos de GIT.....	9
Comandos qué más usaréis.....	9
Buenas prácticas.....	15
Semantic versioning.....	15
¿qué estrategia de tags seguimos?.....	15
¿Cómo hacer un buen commit?.....	16
Primer Commit.....	16
.gitignore.....	16
GIT-FLOW.....	17
¿Qué es?.....	17
¿Quién las inventó?.....	17
¿Por qué se recomienda seguirlas?.....	17
En qué consiste.....	18
Ramas principales de trabajo.....	18
Feature.....	18
Release.....	18
Hotfix.....	19
¿Y todo esto... hay que hacerlo a mano?.....	19
Repositorios GIT.....	20
¿Qué son?.....	20
¿Qué repositorios existen?.....	20
Recordad.....	20
EMPECEMOS A USAR GIT.....	21
¿Existe info buena para formarme más?.....	21
¡Hagamos un ejemplo!.....	22
Clave SSH.....	22
Inicializar un repositorio.....	22
Trabajar con remoto... local.....	22
Feature a lo “clásico”.....	23
Feature con Flow.....	23
Tag de la primera versión.....	23
Distribuimos el código.....	23
Agregar un repositorio remoto... remoto.....	24
Clonar un repositorio remoto.....	24
¡Os toca!.....	24
Primeros merges.....	24
Primeros merges con conflictos.....	24
Entregad vuestra versión.....	24
Y si queréis practicar más.....	25

GIT – mentalización



Qué es, para qué sirve, cómo se usa, y qué mentalidad debemos tener

¿Qué vamos a ver hoy?

- 1.- Qué es git, porqué usarlo, y qué diferencias tiene con SVN
- 2.- ¿Cambio de mentalidad?
- 3.- ¿Qué cliente usar?
- 3.- Comandos básicos
- 4.- Buenas prácticas
- 5.- Probaremos a usar GIT en local y con remotos



De dónde viene GIT

El mantenimiento del núcleo de Linux, durante 1991-2002 se hizo con parches y archivos.

En 2002, el proyecto del núcleo de Linux empezó a usar un DVCS propietario llamado BitKeeper.

En 2005, BitKeeper dejó de ser gratuita.

Linus Torvalds desarrolló su propia herramienta con lo que aprendió de BitKeeper.

OBJETIVOS:

Git es un software de control de versiones diseñado por Linus Torvalds, pensando en la eficiencia y la confiabilidad del mantenimiento de versiones de aplicaciones cuando éstas tienen un gran número de archivos de código fuente.

- Velocidad
- Diseño sencillo
- Fuerte apoyo al desarrollo no lineal (miles de ramas paralelas)
- Completamente distribuido
- Capaz de manejar grandes proyectos

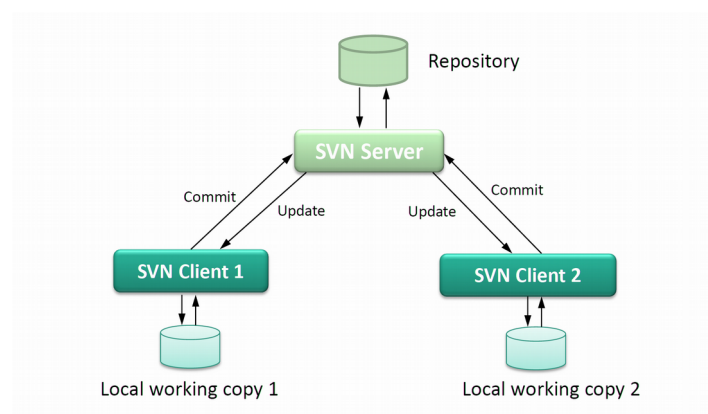


GIT vs SVN



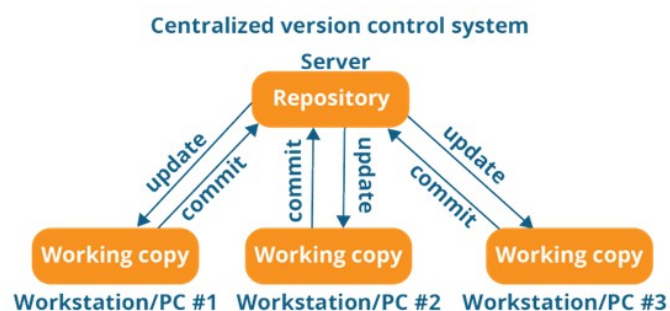
SVN se basa en un sistema de control de versiones centralizado.

Esto significa que existe un almacén central de datos (el repositorio) accesible a todos los usuarios



GIT es un sistema de control de versiones distribuido

Lo que significa que, exista o no un repositorio central, todos los usuarios tienen su propia copia de trabajo. De esta forma, todos tienen acceso al repositorio completo, incluyendo el historial local, sin depender de ningún tipo de conexión de red



GIT para	SVN para
No queramos depender de una conexión de red permanente.	Trabajamos con archivos binarios de gran tamaño.
Queramos tener seguridad en caso de fallo o pérdida de los repositorios principales.	Queremos tener todo el trabajo alojado en un solo sitio.
No tengamos permisos especiales para los directorios.	Queremos poner permisos de acceso a determinadas carpetas de un proyecto
Queramos tener una rápida sincronización de cambios.	Queremos guardar la estructura de los directorios vacíos
Necesitemos tener más tranquilidad con los cambios que realizamos	Podamos realizar cambios en el repositorio sin excesivo riesgo

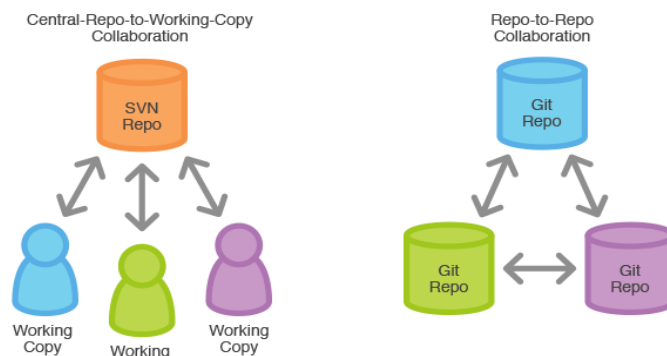
https://www.youtube.com/watch?v=_yQIKEq-Ueg

Video seminario git vs subversión https://www.youtube.com/watch?v=nR5L3sJRp_c (1h29m)

¿Cambio de mentalidad?

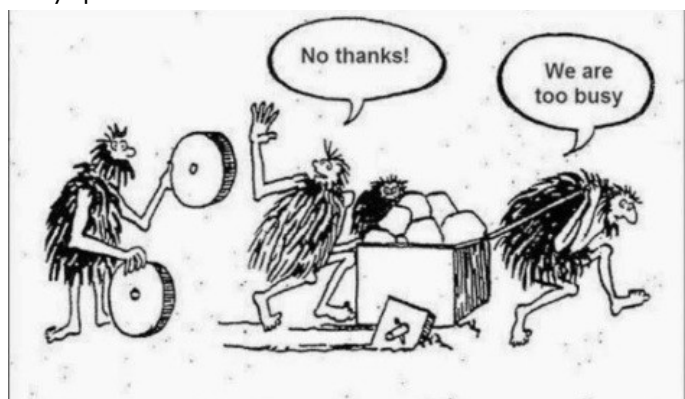
Distribuido

Tenemos que acostumbrarnos a ver GIT como un sistema ¡DISTRIBUIDO!



Tenemos que trabajar con control de versiones

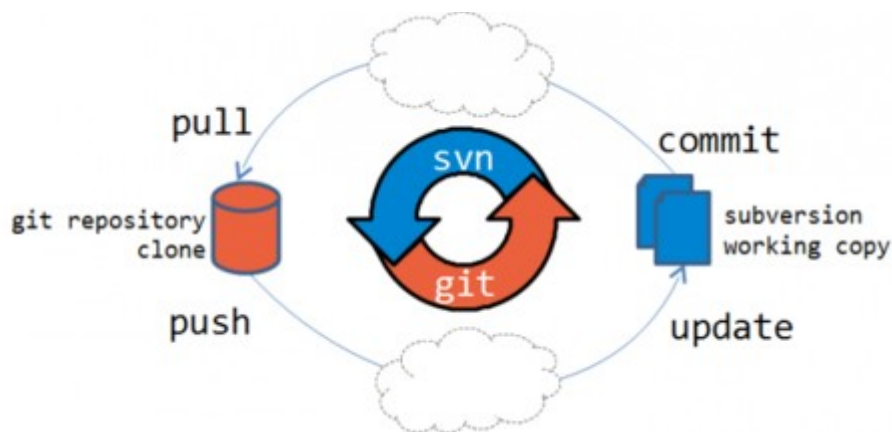
- Tener un buen sistema de versiones
- Poder volver a un punto anterior de forma fácil
- Poder trabajar en equipo
- Saber qué se ha cambiado y quién lo ha cambiado



Git no es SVN

- Liarla con GIT es más difícil
- GIT es distribuido, es decir, si la lías, solo la lías en tu equipo
- ¡Hay más copias!
- Repito, tienes una copia local de todo en tu equipo, ¡pero es una copia!
- Solo un "push" tiene capacidad de liarla, pero aun así es difícil
- Las instrucciones, aunque hay muchas iguales, hacen cosas diferentes

SVN	Git
svn checkout	git clone
svn up	git pull
svn commit	git push
svn log	git whatchanged
svn import	git add -A
svn revert file	git checkout file
svn revert . -R	git reset --hard HEAD



	GIT	SVN
SUBIR cambio a un repositorio	push	commit
DESCARGAR cambios de un repositorio	pull	update

¿Tan difícil es cambiar de subversion a git?

Sí y no

¿Qué problemas me voy a encontrar?

Los que hemos hablado. Distribución, comandos, y miedo

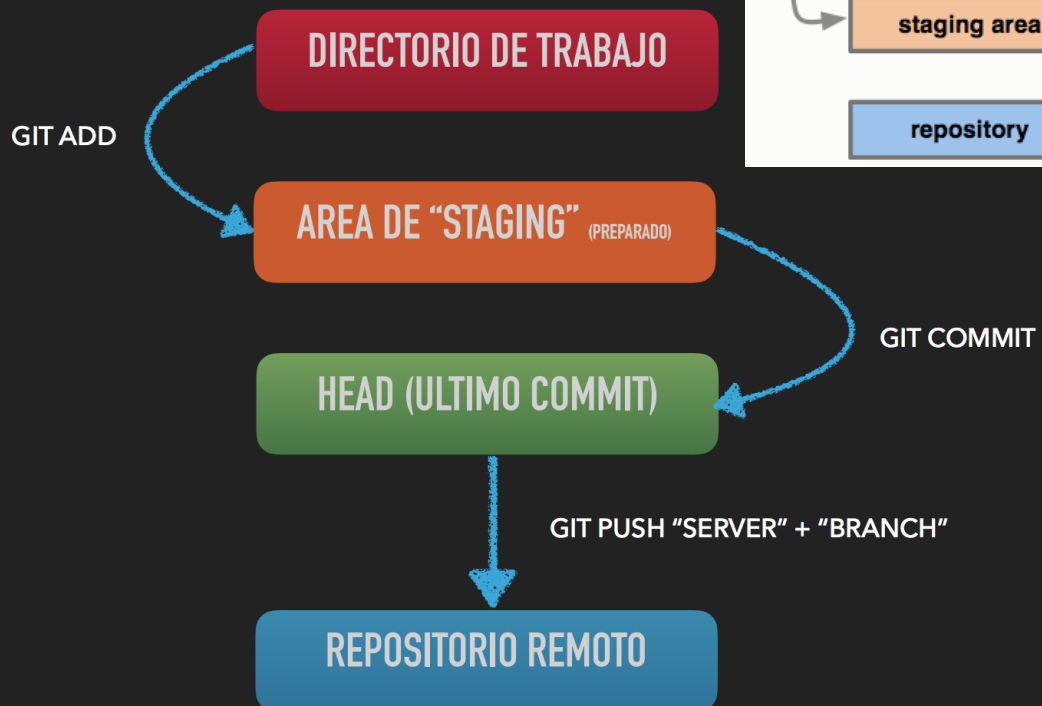
¿Cuánto tiempo me va a costar cambiar la mentalidad?

Yo llevo un año y aún hay cosas que me sorprenden

GIT es simple, cambiar de mentalidad es difícil

Estados de git

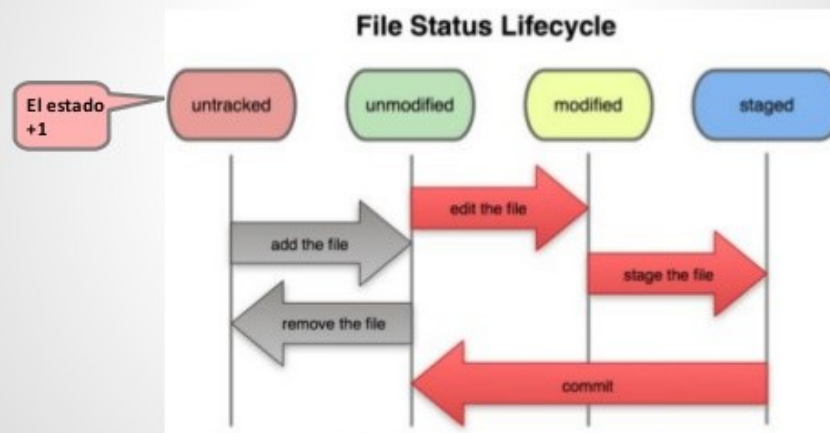
FLUJO DE TRABAJO



Teoría de git

El estado + 1 de Git

Git no sabe qué ficheros existen en el repositorio hasta que se añaden (la primera vez sólo), **untracked**.



Fuente: <http://git-scm.com/book>

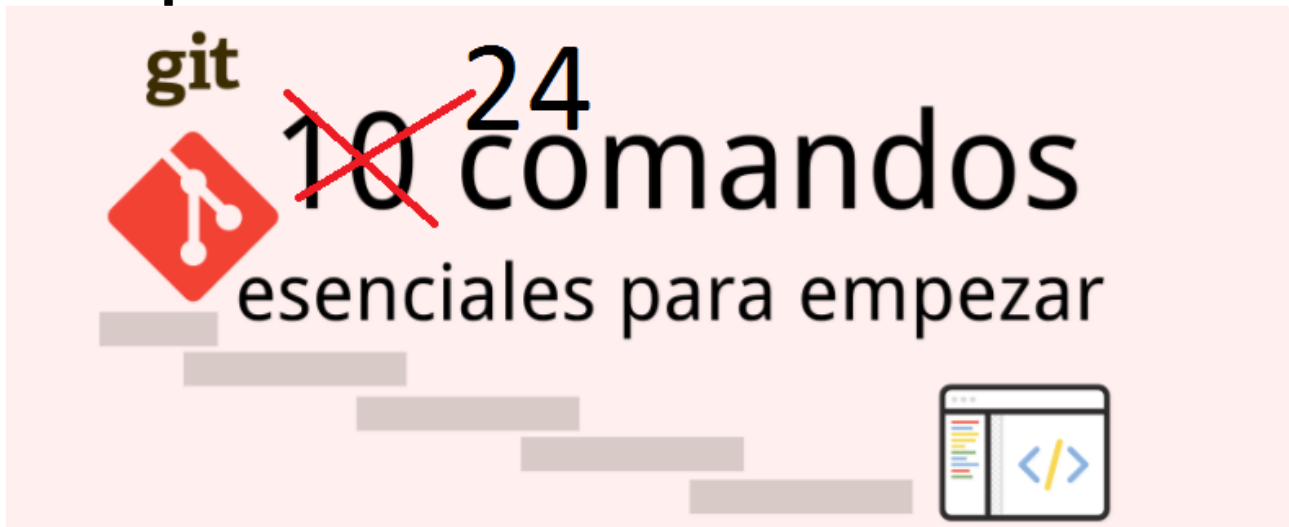
19

¿Qué clientes existen?

- 1.- Git Bash
- 2.- SourceTree
- 3.- TortoiseGit
- 4.- Git for Windows
- 5.- GitHub Desktop



Principales comandos de GIT



<http://viviryaprenderweb.com/10-comandos-git-esenciales-para-saber-por-donde-empezar/>

Comandos que más usaréis

1) git clone

Descargar el contenido de un repositorio a tu máquina.

Automáticamente ese será el repositorio "origin"

`git clone url -b nbRama nbCarpeta`

2) git add

Agregar archivos al Staging Area, es como decirle a git que vigile el estado de unos archivos.

`git add nbArchivo`

`add .` ('archivo punto' para agregar todos) / `add *.txt` (para filtrar los que se quieren agregar)

Eliminar un archivo → `git rm nombreArchivo --cached` (no lo borra del sistema) `-f` (sí lo borra del sistema)

Mover un archivo → Se hace con el sistema operativo

git detecta un archivo borrado y un archivo creado, pero al ver que son iguales, piensa que solo lo has renombrado

3) git commit

Confirmar los cambios para que se almacenen en el repositorio

Un commit son "cambios", por lo que es recomendable hacer commits con cambios específicos y no juntar muchos cambios bajo un mismo commit

`git commit -m "mensaje descriptivo"`

Cambiar el mensaje y archivos del ÚLTIMO commit (agrega archivos del staging area)

`git commit --amend`

In case of fire 



1. `git commit`



2. `git push`



3. `leave building`



4) git push

Enviar cambios a un repositorio remoto

```
git push nbRepositorio nbRama
```

Para fijar un repositorio y una rama usaremos "-u", así no habrá que ponerlo cada vez.

```
git push -u nbRepositorio nbRama
```

Si queremos subir a un repositorio todo nuestro repositorio (ojo, TODAS las ramas y tags, también)

```
git push nbRepositorio --all
```

5) git pull

Sirve para recibir commits de un repositorio

```
git pull nbRepositorio nbRama
```

```
git pull --rebase
```

Con "rebase" podremos reorganizar los commits de forma que evitaremos mergeos innecesarios y por tanto nuevos commits que no aportan información destacable. Hay que tener cuidado porque "rehace" commits copiándolos, así que tendrán un hash nuevo.

6) git init

Inicializar un repositorio

```
git init (crea una carpeta oculta .git)
```

```
git init --bare repositorio.git (lo inicializa sin "work", solo sirve para recibir commits)
```

7) git status

Nos da información sobre el estado de los archivos de nuestra carpeta de trabajo, y del staging area

8) git remote

Un repositorio remoto solo son ramas nuevas que se agregan.

Permite conectarnos con las ramas de ese repositorio.

Se pueden agregar más repositorios remotos con

```
git remote add nbRepositorio url
```

9) git fetch

Descargar los cambios de todas las ramas "remotes/origin/*" a nuestro repositorio.

Así disponemos de los cambios remotos para poderlos mergear si deseamos

10) git branch

Es el comando para trabajar con ramas. Si bien es cierto que "git checkout" lo complementa

- Crear una rama → git branch nbRama (o git checkout -b nbRama)
- Ver las ramas de un proyecto → git branch (git branch -all para ver también las ramas remotas)
- Renombrar una rama → git branch -m nbRamaVieja nbRamaNueva
- Borrar una rama → git branch -d nbRama
- Cambiar de rama → git checkout nbRama

11) git checkout

Permite realizar acciones sobre la "fotografía" actual de nuestro repositorio

Cambiar de rama

git checkout nbRama

Deshacer un cambios de un archivo "controlado", que aún no se hayan "commiteado", es decir, descargar las modificaciones

git checkout -- nbArchivo (si no se pone nada, es para todos)

Volver a un determinado punto, para trabajar a partir de entonces

git checkout hash

¿Por qué se puede volver a un determinado punto, o cambiar de rama con la misma instrucción?

Porque checkout en realidad lo que hace es cambiar el "puntero" que nos da el estado actual.

Por eso, podemos apuntar a una rama u otra, o podemos apuntar a un commit del pasado.

12) git log

Examinar el registro de cambios

La información que da es:

- primera línea identificador hash largo (40 caracteres) pero de normal solo se ven los primeros 7 caracteres
- segunda línea, autor
- tercera línea, fecha del commit
- resto de líneas, mensaje del commit

git las opciones que nos da:

- --oneline para verlo formateado en una línea
- --decorate nos la informacion de los punteros
- --graph nos da informacion gráfica
- --all para ver todos los commits

13) git diff

Nos indica qué archivos se han modificado en nuestro staging area respecto a su último commit

También podemos ver los cambios entre 2 commits

git diff hash_inicial hash_final

14) git whatchanged

Es como git log, pero indicándonos los archivos que se han modificado en cada commit

15) git show

Es como git log, pero de un commit/tag/rama específico

16) git merge

Sirve para fusionar ramas. Desde la rama a la que queremos incorporar los cambios, llamamos a la rama de la que nos los queremos traer
git merge feature/nbRama

Git nos da la opción que ya hemos visto de "git pull".

"Pull" es básicamente un "fetch + merge". Es decir, se trae lo cambios de las ramas remotas, y los mergea en nuestra rama actual.

Si queremos obligar a que en caso de conflicto se resuelvan automáticamente, podemos usar "estrategias de mergeo"

git merge feature/nbRama -s ours/theirs

Como hace los merge git

- Con diferentes archivos

git va bien. Utiliza la estrategia "fast-forward", mueve el puntero al último commit.

- Con mismos archivos, pero con cambios en diferentes partes

git suele ir bien. Utiliza una estrategia recursiva

- Con mismos archivos y misma región de código

git genera un conflicto

```
<<<<<< HEAD
codigo1
=====
codigo2
>>>>>> nombre rama
```

Una vez resuelto hay que añadir el archivo "git add nbArchivo" y commitarlo "git commit"

La estrategia "recursiva" siempre crea un tercer commit, o automático por git, o manual por el usuario tras la resolución de un conflicto.

En caso de que no sepamos qué hacer, podemos salir del "estado de mergeo" dejando todo como estaba
git merge --abort

17) git blame

Nos dice quien ha modificado cada linea de un archivo, indicándonos incluso el commit al que pertenece el cambio

18) git cherry-pick

Fusiona en nuestra rama actual solo el "commit" que le indiquemos.

19) git stash

Usa una lista LIFO para almacenar cambios en el "staging area" que aún no podemos commitear.

Al introducir un cambio es esta pila, los archivos físicos "pierden" los cambios porque estos se almacenan.

- o git stash save "mensaje descriptivo" (el save es opcional)
- o git stash list
- o git stash apply → aplica los cambios que hay en el último stash
- o git stash drop → borrar la pila
- o git stash pop → saca el ultimo que ha metido (apply + drop)
- o git stash branch nbRama idStash → genera una rama con los stash indicado
- o git stash pop stash@\{1\}

20) git reset

Deshacer cambios de forma DESTRUCTIVA (es peligroso)

Se usará para borrar un commit.

IMPORTANTE: sin que se haya "distribuido" este commit!!!!

Siempre indicaremos la posición que se queda fija, y desde esa(que no se borra) se eliminarán todos los commits más recientes.

Posicion = <hash>, <HEAD>, o a <hash|HEAD~numeroHaciaAtras>

3 tipos de reset:

git reset --soft pos → elimina los commits, deja el contenido preparado para añadir

git reset pos → elimina los comits, deja untracked los cambios que había

git reset --hard pos → elimina commits y cambios, como si no hubiese existido!

Podemos usarlo también para sacar del staging area un fichero, sin deshacer los cambios

git reset HEAD nombreArchivo

Con "git reset HEAD --" sacamos todo lo que hay en el staging area

Realiza la misma función que

git rm nombreArchivo --cached (con -f se sacaba y se borraba!)

21) git revert

Revert te permite deshacer cambios de una forma NO TAN DESTRUCTIVA

Lo que hace en realidad es aplicar el opuesto de un commit, para "neutralizarlo" con el nuevo commit.

Es decir, que agrega un nuevo commit siempre

git revert HEAD o hash7

¿Y si queremos hacer varios revers, pero no dejar tantos commits?

git revert -n HEAD o hash7

usar tantas veces como haga falta, y cuando se hayan terminado de revertir cambios, se hace:

git revert --continue (si queréis salir sin hacer el revert --abort)

y a continuación os pedirá el mensaje para el commit

22) git rebase

Rebase es un COMANDO DESTRUCTIVO y hay que tener cuidado con él. Aplicar solo a cambios no "distribuidos"

Rebase "aplana" los cambios reorganizando los commits de forma que no haga falta incluir "commits de mergeo"

git rebase ramaPadre (estando en una rama) aplana los commits dejando un solo hilo y "copiando" los commits fusionados. Es decir, tendrán un nuevo hash.

Rebase interactivo sirve para combinar commits, cambiar el orden, los mensajes...

git rebase -i HEAD~4 (jugaremos con los últimos 4)

Si cambiamos el orden de los commits, en realidad no cambia la fecha de ejecución, por lo que podemos formar un pequeño lio

23) git tag

Un tag es una etiqueta que hace más fácil poder llegar a un determinado commit

Existen los tags ligeros y los anotados

- ligeros
 - Las marcas ligeras suelen usarse para indicar versiones variadas
 - `git tag nombre` (si no se indica nada, usa la posición actual, la HEAD)
 - `git tag nombre hash7` (marca la posición del commit indicado)
- tags anotados
 - Son para cosas más serias, ya que no solo realizan la marca, si no que también guardan un objeto con la información
 - `git tag -a v0.1.0`

Otros comandos para los tags son:

`git tag -d nbTag` → elimina el tag

`git tag` → lista todos los tags

`git tag -l` → con `-l ""` puedes filtrar, por ejemplo `-l "v2.0.*"`

24) git config

Sirve para configurar git

`git config --global user.name "Chabier"`

`git config --global user.email "<fjnoqueras@indra.es>"`

Se puede usar también para crear un alias (una forma fácil para escribir menos)

si agregas `--global`, se aplica a todos repositorios

`git config alias.NbAtajo "log --oneline --decorate --all --graph"`

Un alias útil: `alias.unstage 'reset HEAD --'` quita todo lo que hay en el stage

al alias se le pueden seguir pasando parámetros

`git config --global --get-regexp alias` → muestra todos los alias creados

`git config --global --unset alias.logdag` → borra un determinado alias

lista de videos de un tutorial en castellano sobre git

<https://www.youtube.com/playlist?list=PLTd5ehIj0goMCnj6V5NdzSIHBgrIXckGU>

Buenas prácticas

Semantic versioning

¿qué estrategia de tags seguimos?

¿Cómo hacer un buen commit?

Primer Commit

.gitignore

git flow

BUENAS PRÁCTICAS



Semantic versioning

<https://semver.org/>



¿qué estrategia de tags seguimos?

Los tag nos sirven para indentificar versiones definitivas o destacables

tag ligeros

git tag v1.2.0 9fceb02

tag anotados

git tag -a v1.2.0 -m 'version 1.2' 9fceb02

<https://git-scm.com/book/en/v1/Git-Basics-Tagging>

¿Cómo hacer un buen commit?

Un commit suele tener esta forma:

```
git commit -m "mensaje descriptivo del commit"
```

Pero un buen commit tiene 3 partes

1) Primera línea : ¿Qué hace el commit?

explicación de lo que hace, en imperativo y con menos de **50 caracteres**

2) Dos bloques con **72 caracteres** como máximo por línea

2.1) Explicación de forma más general y porqué lo hemos hecho

2.2) Explicación más detallada y qué esperamos del commit

```
#####
```

Implementa una mejora en el listado de facturas.

Con esta modificación hemos agregado una cabecera dinámica a la tabla que da la opción de organizar la tabla según la ordenación de la columna que el usuario desee.

Hemos agregado la librería de jQuery <datatables> que nos permite configurar cualquier tabla de la aplicación para hacerla dinámica haciendo referencia a ella con <\$('#nbTabla').DataTable();>. Esto modificará la vista de la tabla en HTML implementando una cabecera javascript que permitirá al usuario ordenar por la columna que desee

```
#####
```

<https://codigofacilito.com/articulos/buenas-practicas-en-commits-de-git>

Primer Commit

Como consejo, cuando inicies un proyecto, hacer un primer commit con un archivo de inicio, por ejemplo un "README.md" que contenga el nombre del proyecto.

Os evitará problemas futuros, tan nimios pero tan importantes como poder volver a empezar desde el primer commit.

Además este paso suele ir a la vez que el siguiente

.gitignore

No queremos que git "vea" todo el sistema de archivos, ya que hay algunos que no nos interesan, por ejemplo, los archivos compilados.

Para eso, bastará con añadir .gitignore en la raíz del proyecto

Aquí tenemos un ejemplo que nos ayuda a ignorar ficheros

<https://gist.github.com/octocat/9257657>

GIT-FLOW



¿Qué es?

git-flow son unas "reglas del juego" y un plugin de git

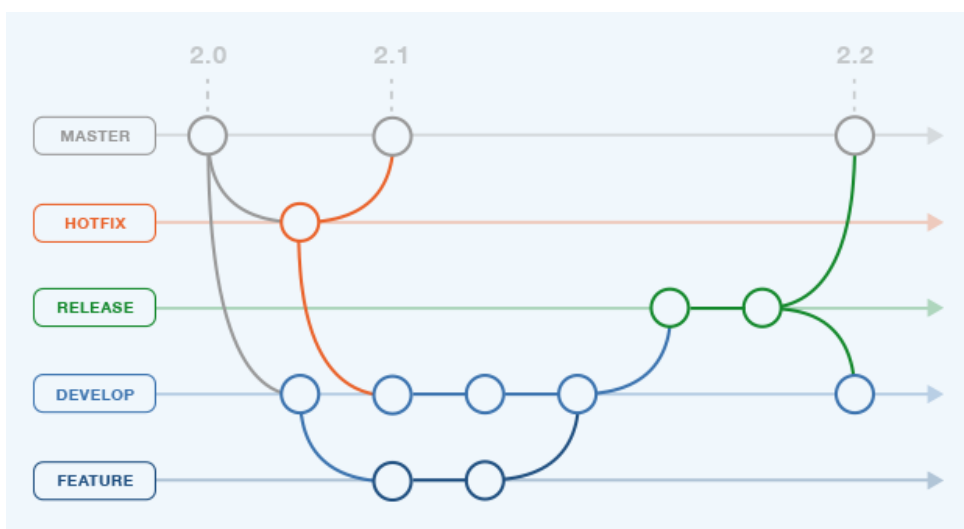
¿Quién las inventó?

Vincent Driessen, que en enero de 2010 publicó en su blog la entrada

"A successful Git branching model" → <http://nvie.com/posts/a-successful-git-branching-model/>

¿Por qué se recomienda seguirlas?

Porque aunque no son reglas absolutas, en la mayoría de casos nos da una estrategia de trabajo que puede ayudar en el mantenimiento de un proyecto.



En qué consiste

Ramas principales de trabajo

"master": cualquier commit que pongamos en esta rama debe estar preparado para subir a producción

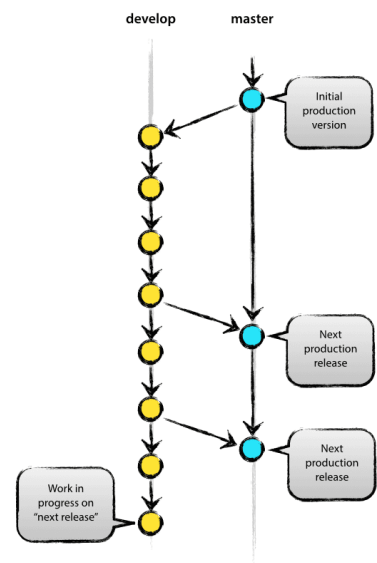
"develop": rama en la que está el código que conformará la siguiente versión planificada del proyecto

Ramas auxiliares de trabajo

feature

release

hotfix



Feature

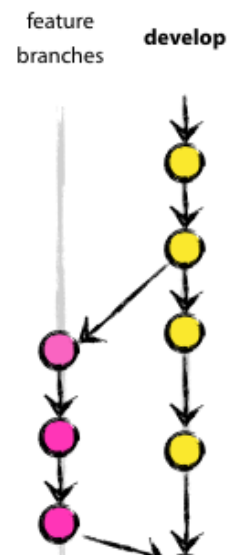
Estas ramas se utilizan para desarrollar nuevas características de la aplicación

- Se originan a partir de la rama develop.
- Se incorporan siempre a la rama develop.

Nombre:

cualquiera que no sea master, develop, hotfix-* o release-*

Se recomienda "feature/nombreRama"



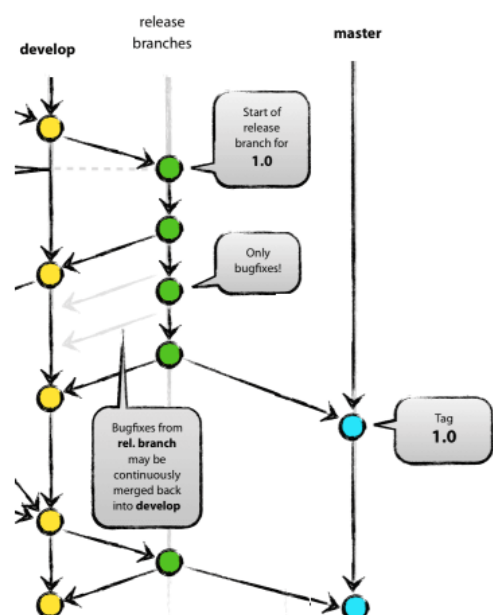
Release

Estas ramas se utilizan para preparar el siguiente código en producción. En estas ramas se hacen los últimos ajustes y se corrigen los últimos bugs antes de pasar el código a producción

- Se originan a partir de la rama develop
- Se incorporan a master y develop

Nombre: release-*

Se recomienda "release/nombreRama"



Hotfix

Esas ramas se utilizan para corregir errores y bugs en el código en producción. Funcionan de forma parecida a las Releases, siendo la principal diferencia que los hotfixes no se planifican.

- Se origina a partir de la rama master
- Se incorporan a la master y develop

Nombre: hotfix-*

Se recomienda "hotfix/nombreRama"



¿Y todo esto... hay que hacerlo a mano?

¡No!

Git ya trae instalado por defecto este plugin, y podemos usarlo para facilitarnos la vida.

Una vez estemos en nuestra carpeta del repositorio, iniciamos "Flow"

`git flow init` (con `-d` si no queremos que haga muchas preguntas)

Iniciar una rama, y mergearla donde corresponde con solo 2 comandos

`git flow feature start nombre_de_funcionalidad`

`git flow feature finish nombre_de_funcionalidad`

Y además, para poder ver las ramas feature creadas

`git flow feature`

Los mismos comandos para release y hotfix

Más info en:

<http://codeloveandboards.com/blog/2013/04/06/automatiza-tu-manera-de-usar-git-con-git-flow/>

Repositorios GIT

¿Qué son?

Tenemos que partir de la base de que no existen "servidores" o "repositorios" de GIT.

Los "repositorios" en realidad son otros nodos que tienen un cliente más bonito.

¡GIT ES DISTRIBUIDO!

¿Qué repositorios existen?

Sin embargo, podemos considerar "nodos repositorios" algunos tales como estos:



Recordad



¿Sólo esto nos cuentas de los repositorios?

Bueno, vamos a agregar que puedes tener una especie de repositorio en local iniciando un proyecto de git sin carpeta de trabajo

```
git init --bare
```

Pero en realidad, cualquier nodo puede ser un repositorio.

EMPECEMOS A USAR GIT

Instalaremos git desde

<https://git-scm.com/downloads>



¿Existe info buena para formarme más?

Ejemplos online que podemos seguir

<https://try.github.io/>

Uno bastante bueno

<https://learngitbranching.js.org>

Tutorial de git no interactivo

<http://es.gitready.com>

Formato físico-digital

LIBRO : PRO GIT (gratis en kindle)

<https://www.amazon.es/Pro-Git-Scott-Chacon-ebook/dp/B004TTXLGI>

¡Hagamos un ejemplo!

Clave SSH

Vamos a hacernos una clave ssh para trabajar más cómodos

```
cd ~/.ssh
```

```
ssh-keygen
```

editar "config" y agregar

```
Host github.com
```

```
    Hostname github.com
```

```
    User git
```

```
    IdentityFile ~/.ssh/github
```

y enviar el contenido del archivo .pub

Inicializar un repositorio

Creo una carpeta donde voy a inicializar mi repositorio

```
mkdir practicaTest
```

```
git flow init -d
```

```
git branch -all
```

agregamos README.md y .gitignore

```
git status
```

```
git add .
```

```
git commit
```

Trabajar con remoto... local

Creo una carpeta donde voy a inicializar mi repositorio local

¡Cualquier carpeta proyecto de git es un repositorio!

¡Pero para este ejemplo vamos a usar la opcion --bare para darle más apariencia

```
mkdir miRepositorioGit
```

```
cd miRepositorioGit
```

```
git init --bare
```

Y ahora, agregamos el repositorio a nuestro proyecto

```
git remote add repoLocal ../miRepositorioGit
```

y pusheo los primeros cambios

```
git push -u repoLocal develop
```

Feature a lo "clásico"

```
git checkout -b feature/pagina_inicio
```

(agregamos los archivos)

```
git add .
```

```
git commit -m "crea la pagina de inicio de nuestra web de pruebas"
```

```
git checkout develop
```

```
git merge feature/pagina_inicio
```

Feature con Flow

```
git flow feature start lista_chula
```

(agregamos los archivos)

```
git add .
```

```
git commit -m "crea la pagina de inicio de nuestra web de pruebas"
```

```
git flow feature finish -k lista_chula
```

Tag de la primera versión

De forma clásica

Tagamos el resultado para entregar una release

```
git tag -a v0.1.0-tradicional -m "primera version de nuestra web"
```

De forma con flow

```
git flow release start v0.1.0
```

(agregamos un último cambio)

```
git flow release finish v0.1.0
```

Distribuimos el código

Subimos todos los cambios en código, ramas y tag al repositorio

```
git push --all
```


Agregar un repositorio remoto... remoto

Agrego un remoto

```
git remote add github git@github.com:chabierzgz/PracticaIndra.git
```

Clonar un repositorio remoto

Y para empezar a trabajar, vamos a clonar un repositorio remoto

Dejad en el terrible olvido lo que habéis hecho ahora

Y descargad el siguiente repositorio

```
git clone https://github.com/chabierzgz/practicaTest.git -b develop
```

Importante: si queréis usar git flow, tenéis que iniciarlo "git flow init -d"

¡Os toca!

Generar una feature para incluir vuestra página SOLO LA PAGINA, no la enlacéis aún

Hacedla, mergeadla, commiteadla y subidla a git a "develop"

Cuando todos la tengamos

Descargar los cambios (git pull)

Como cada uno habéis tocado un archivo independiente, no debería haber ningún problema

Primeros merges

Descargar los cambios (git pull)

Cread una feature para incluir el enlace a vuestra página en index.html

Esperad a que yo haga un cambio en index.html

Haced el cambio en index.html indicando solo la url a vuestra pagina

Terminad la feature, mergearla, y subid los cambios

Primeros merges con conflictos

Descargar los cambios (git pull)

Cread una feature para agregar vuestro email al nombre en página en index.html

Esperad a que yo haga un cambio en index.html (poner una marca de -email aqui-)

Haced el cambio en index.html

Terminad la feature, mergearla, resolved el conflicto y subid los cambios

Entregad vuestra versión

Cread una entrega de la vesion v0.1.1-nombre

Finalizad la entrega

Subid el tag con "git push --tags"

Terminad la feature, mergearla, resolved el conflicto y subid los cambios

Y si queréis practicar más...

Recodad, este tutorial es bastante bueno

<https://learngitbranching.js.org>

